STAT:5400 Tech Guide 1 — Linux and the lnx Machines

Boxiang Wang

2026-10-26

1 UNIX and Linux

- In 1969, Ken Thompson and Dennis Ritchie of Bell Labs developed the UNIX operating system. At that time, every model of computer had a different operating system.
- The **kernel** is the core of UNIX. It controls how the hardware interacts with the software. All other functions are built around the kernel.
- UNIX was later rewritten in C. Thus UNIX became compatible with many hardware architectures.
- UNIX was first used only in large computing environments like universities, government, and large companies. macOS is a UNIX-based system.
- UNIX is not free. Richard Stallman decided to create a free UNIX-like system, GNU (GNU's Not UNIX). Under the GNU license, everyone has the freedom to use, copy, and modify the software.
- Linus Torvalds developed the Linux kernel in 1991 and used the GNU license.
- Because of the free nature, many operating systems have been distributed based on the Linux kernel. Free ones include Ubuntu, Debian, and Fedora. Commercial ones include Red Hat Enterprise Linux. Android is also based on Linux.
- Nowadays, Linux runs on a wide range of hardware: personal machines, phones, servers, and cloud services. The lnx machines, IDAS, and Argon all run Linux.
- Linux is not on the midterm.

2 Remote access to the lnx machines

The department has Linux servers `l-lnx200` to `l-lnx217`. You log in with your HawkID and its password.

- For Mac users, open Terminal. For Windows users, open PowerShell. Then type

```
ssh hawkid@l-lnx200.divms.uiowa.edu
```

- If one machine does not answer, try the next number.
- Off campus, you need the VPN first.
- FastX gives a Linux desktop in the browser. Log in with your HawkID and click the plus sign to start a new session.
- If you write a file on your own computer, use the secure copy `scp` to upload it to the server. Run this on your own computer, not on the server:

```
scp TypeIErr2.R hawkid@l-lnx200.divms.uiowa.edu:~/5400/
```

- The Codespace of this course also has a terminal (menu Terminal, New Terminal). The commands on this page work there. The `ssh` and `scp` lines do not, because the Codespace is outside the campus network.
- If you have issues with your Linux account, visit https://clas.uiowa.edu/linux/help.

3 Shell

- The **shell** is a program that reads and runs your command. The departmental servers use the Bourne Again Shell (bash). macOS uses the Z shell (zsh) by default. For this page they are the same.
- To run the shell, you need a **terminal**. With a terminal open, you see a blinking cursor. This is the command line where you type.

```
echo $SHELL
```

Tips:

- In the shell, use the up arrow to go back to the command you just typed.
- Use the Tab key to complete a command or a file name.
- Use `Ctrl + A` to move the cursor to the beginning of the line and `Ctrl + E` to the end.
- Use `Ctrl + C` to stop a running command.

You may try these commands to get started:

```
date
cal
whoami
hostname
```

On Windows, these commands need a bash shell. Install Git for Windows, which comes with one. We do not use Git today.

4 Filesystem

- The **filesystem** is how the files are organized on the disk. The Linux filesystem is a single tree. It begins at the root directory `/`.
- Linux uses the forward slash `/` instead of the backslash `\` as in Windows.
- `a/b` is the file or directory `b` inside the directory `a`.

Symbol	Path
`/`	the root directory
`~`	your home directory
`.`	the current directory
`..`	the parent of the current directory

For the rest of this page, make a directory for practice and go there:

```
mkdir -p ~/STAT5400/linux
cd ~/STAT5400/linux
```

5 Basic commands

5.1 `pwd`, `ls`, and `cd`

`pwd` prints the working directory. `ls` lists the contents of a directory. It is perhaps the most commonly used command in Linux. `cd` changes the directory.

```
pwd
ls
ls -l
ls -a
cd ..
pwd
cd -
```

- `ls -l` gives a long list with more details. `ls -a` includes hidden files, whose names start with a dot. You can also do `ls -la`.
- `cd ..` goes to the parent directory. `cd ~` or `cd` alone goes to the home directory. `cd -` goes back to the previous directory.

5.2 `mkdir` and `touch`

`mkdir` makes directories. `touch` creates an empty file.

```
mkdir dir1 dir2
touch a.txt b.txt c.txt
ls
```

Do not use a space in a directory name.

5.3 `cp`, `mv`, and `rm`

`cp` copies. `mv` moves. `mv` can also be used to rename a file. `cp -r` copies a whole directory.

```
cp a.txt a1.txt
mv b.txt dir1/
mv a1.txt a2.txt
cp -r dir1 dir3
ls
ls dir1 dir3
```

`rm` removes files and directories.

```
rm a2.txt
rm dir2
rmdir dir2
rm -r dir3
ls
```

- `rm dir2` fails, because `dir2` is a directory. `rmdir` removes an empty directory. `rm -r` removes a directory with everything inside.
- `rm -rf` removes everything regardless of being protected. Be very careful with this command. There is no trash can in Linux.

5.4 `cat`, `less`, `head`, and `tail`

We first create a text file with the integers 1 to 100, one number per line:

```
for i in {1..100}; do echo $i; done > numbers.txt
```

- `cat` displays the whole file. Use it when the file is short.

- `less` displays the file page by page. Use the arrow keys to move. Type `q` to quit.
- `head -n 4` prints the first 4 lines. `tail -n 4` prints the last 4 lines.
- `wc` counts lines, words, and characters.

```
head -n 4 numbers.txt
tail -n 4 numbers.txt
wc numbers.txt
```

The output of `wc` is `100 100 292 numbers.txt`. The file has 100 lines, 100 words, and 292 characters.

5.5 `sort` and `grep`

`sort` sorts the lines of a file. `grep` prints the lines that match a pattern.

```
sort numbers.txt | head -n 5
sort -n numbers.txt | head -n 5
grep 5 numbers.txt
```

Note that `sort` sorts the lines as text, so `10` comes before `2`. `sort -n` sorts them as numbers.

5.6 `man` and `history`

`man` opens the manual of a command. It is similar to `help` in R. `history` lists the commands you used.

```
man ls
history
```

Type `q` to leave the manual. You may also try `ls --help`.

6 Pipelines and redirection

- The pipe `|` chains two commands. The output of the first command is the input of the second one. Usage: `command1 | command2`.

```
ls | sort -r
grep 5 numbers.txt | wc -l
```

- `>` writes the output to a file. `>>` appends the output to the end of a file.

```
echo "STAT:5400" > course.txt
cat course.txt
echo "Statistical Computing and AI" >> course.txt
cat course.txt
echo "Statistical Computing and AI" > course.txt
cat course.txt
```

Note that `>` replaces the file. The last `cat` shows one line only.

7 Shell variables and metacharacters

```
courseID=5400
echo $courseID
echo "The course is $courseID"
echo 'The course is $courseID'
echo $HOME
```

- No spaces around `=`. Use `$courseID` to read the variable. Use `unset courseID` to delete it.
- Double quotes fill in the variable. Single quotes keep the text as it is.
- `$HOME` is your home directory. `$PATH` is the list of directories where the shell looks for commands.

Metacharacters:

- `#` starts a comment.
- `;` separates two commands on one line.
- `*` matches any number of characters in a file name. `?` matches one character.

```
ls *.txt
ls ?ourse.txt
ls /bin/b*
```

8 File permissions

Unlike Windows, Linux is a multi-user operating system. It has a better control over file security, so that a confidential file is not seen by other users.

```
ls -l numbers.txt
```

The first part of the output looks like `-rw-r--r--`. The security mode of a file has nine characters:

- three sets: user, group, others;
- three permissions: `r`, read; `w`, write; `x`, execute.

So `-rw-r--r--` means the user can read and write, the group can read, and others can read.

Change file permissions by `chmod`:

- the first argument specifies which set to modify: `u` user, `g` group, `o` others, `a` all;
- the second argument is `+` to add permissions or `-` to take them away;
- the third argument is which permission to change.

Examples:

- `chmod g+w numbers.txt` gives the group write permission for this file.
- `chmod a+rx dir1` gives all users read and execute permission for the directory `dir1`.

You may also use the numerical format. `chmod 754 numbers.txt` gives read, write, and execute permissions to the user, read and execute to the group, and read to others.

> **i** The numerical format
>
> **Each digit is a sum: read 4, write 2, execute 1.**
>
> | Value | Permissions | Symbol |
> |---|---|---|
> | 0 | no permissions | `---` |
> | 1 | execute only | `--x` |
> | 2 | write only | `-w-` |
> | 3 | write and execute | `-wx` |
> | 4 | read only | `r--` |
> | 5 | read and execute | `r-x` |
> | 6 | read and write | `rw-` |
> | 7 | read, write, and execute | `rwx` |
>
> `chmod 600 file` keeps a file private: only you can read and write it.

8.1 A shell script

A shell script is a text file of commands. The first line `#!/bin/bash` says which shell runs it.

```
printf '%s\n' '#!/bin/bash' 'echo "STAT:5400 is fun!"' > myscript.sh
cat myscript.sh
ls -l myscript.sh
./myscript.sh
```

The last line gives `Permission denied`. A new file has no execute permission. We add it and run the script again:

```
chmod +x myscript.sh
ls -l myscript.sh
./myscript.sh
```

Below is a shell script that prints all prime numbers between 2 and 50. It shows a `for` loop and an `if` in bash.

```
for num in {2..50}; do
  prime=1
  for ((i = 2; i <= num / 2; i++)); do
    if [ $((num % i)) -eq 0 ]; then
      prime=0
      break
    fi
  done
  if [ $prime -eq 1 ]; then
    echo $num
  fi
done
```

Bash can do this, but it is slow. For statistical work, we write the loop in R and use the shell only to start R.

9 Text editors

9.1 nano

Use `nano` to create a simple text file.

- To create a new file, type `nano hello.txt`.
- Type your text.
- Press `Ctrl + O` to save and `Ctrl + X` to exit. The commands are listed at the bottom of the screen.
- You may check your new file using `cat` or `less`.

9.2 Vim

Vi is a classic editor on Linux. Vim is "Vi improved". Vim is controlled by the keyboard and has a steep learning curve. It is sometimes very convenient to use Vim to edit a file directly on the server.

Vim has two modes: command mode and insert mode. Use `i` to enter insert mode and `Esc` to go back to command mode.

- Open a file: `vim hello.txt`. You start in command mode.
- Type `:set number` to see the line numbers. Type `:5` to move to line 5.
- Use `x` to delete a character. Use `dd` to delete a line. Use `u` to undo.
- Type `:wq` to write and quit. Type `:q!` to quit without saving.

Type `vimtutor` in a terminal for a 30-minute tutorial. A cheat sheet: https://vim.rtorr.com/.

10 Batch execution of R

Let us write an R script based on the example of computing the Type I error that we talked about in Section 3.3. We also let the script read `n` and `dist` from the command line.

Paste the whole block into the terminal. The first line, `cat > TypeIErr2.R <<'EOF'`, writes everything up to the line `EOF` into the file `TypeIErr2.R`.

```
cat > TypeIErr2.R <<'EOF'
pvalDist2 <- function(n, dist, mu_0, alp = 0.05, B = 10000, ...) {
  # simulate data from user-specified dist
  Xlist <- do.call(paste0("r", dist), list(B * n, ...))
  Xlist <- matrix(Xlist, B, n)
  # compute t-statistic for each sample
  Xbarlist <- rowMeans(Xlist)
  sdlist <- apply(Xlist, 1, sd)
  t_stat <- (Xbarlist - mu_0) / (sdlist / sqrt(n))
  # returns the observed p-values
  pt(-abs(t_stat), n - 1) + 1 - pt(abs(t_stat), n - 1)
}
```

```
# read the arguments, e.g.  Rscript TypeIErr2.R 'n=10; dist="exp"'
args <- commandArgs(TRUE)
if (length(args) == 0) stop("No arguments supplied.")
for (a in args) eval(parse(text = a))
if (!exists("n")) stop("n is missing")
if (!exists("dist")) stop("dist is missing")

set.seed(5400)
alp <- 0.05
r2 <- pvalDist2(n = n, dist = dist, mu_0 = 2, alp = alp, rate = 1/2)
TypeIerr <- mean(r2 < alp)
cat("n =", n, " dist =", dist, " Type I error =", TypeIerr, "\n")
save(TypeIerr, file = paste0("TypeIerr_", n, "_", dist, ".RData"))
EOF
```

If you wrote the file on your own computer, upload it to the server with `scp`. Then connect to the server and run R in batch:

```
nice -n19 Rscript --no-save --no-restore TypeIErr2.R 'n=10; dist="exp"'
ls *.RData
```

- The output is `n = 10  dist = exp  Type I error = 0.1003`, and a file `TypeIerr_10_exp.RData` is generated.
- `Rscript` runs the R file without opening R. The text in quotes is passed to the script. `commandArgs(TRUE)` returns it, and `eval(parse(text = a))` runs it as R code, so `n` and `dist` exist in the script.
- The option `nice -n19` lowers the priority of your job. It is required when you run a large job on a shared machine. The options `--no-save --no-restore` tell R not to save or restore the workspace.
- Try `n=50`. The file name changes with the arguments, so the runs do not overwrite each other.

To read the result later:

```
Rscript -e 'load("TypeIerr_10_exp.RData"); TypeIerr'
```

An alternative is `R CMD BATCH`. It writes the code and the output into `TypeIErr2.Rout`:

```
nice -n19 R CMD BATCH --no-save --no-restore '--args n=10 dist="exp"' TypeIErr2.R
```

10.1 Several jobs at once

To run several jobs at the same time, we add `&` at the end of the command. The shell starts the job in the background and goes to the next line without waiting. `wait` waits until all background jobs are done.

```
for n in 10 20 50 100 200; do
  nice -n19 Rscript --no-save --no-restore TypeIErr2.R "n=$n; dist=\"exp\"" &
done
wait
ls *.RData
```

- The five runs use five cores at the same time. `nproc` prints the number of cores of the machine.
- Open a second terminal and type `top` while they run. `top` lists the running processes, the busiest first. You will see five `R` processes. Type `q` to quit `top`.
- Each run saves its own file. To combine them in R:

```
nlist <- c(10, 20, 50, 100, 200)
files <- paste0("TypeIerr_", nlist, "_exp.RData")
res <- sapply(files, function(f) { load(f); TypeIerr })
names(res) <- nlist
res
```

In Tech Guide 2, we do the same from inside R.

> **i** Keeping a job alive after you log out
>
> **A job is killed when you close the connection. `nohup` keeps it alive.**
>
> ```
> nohup nice -n19 Rscript TypeIErr2.R 'n=100; dist="exp"' > run.log 2>&1 &
> ```
>
> `> run.log` sends the output to the file `run.log`. `2>&1` sends the error messages there too. The `&` at the end gives you the prompt back. Another tool is `tmux`: type `tmux new -s job` to open a session, start your job, press `Ctrl + B` and then `D` to leave, and type `tmux attach -t job` to come back later.

11 Exercises

✋ By hand

1. List all the files in `/bin`. Try `ls /bin/*r` to list the files ending in `r`, and `ls /bin/b*` to list the files beginning with `b`.
2. Type `cd` to go to the home directory. Make a new directory `5400`. Inside it, make a directory `tmp`.
3. Create an empty file `my.sh` in `tmp`. Copy it to `5400`. Delete the original file in `tmp`.
4. In `5400/my.sh`, put the two lines `#!/bin/bash` and `echo "STAT:5400 is fun!"`. Change the file permission and run the script.
5. Try `echo $HOME`. Print the sentence `My home directory is /space/hawkid` by completing an `echo` command. Then write the sentence into a file `home.txt`.
6. Print all even numbers between 1 and 20 with a `for` loop.
7. Upload `TypeIErr2.R` to an lnx machine with `scp`. Run it there with `n=30` and `dist="exp"`. Load the saved file in R and print the Type I error.

12 References

- Rav Ahuja, Sam Prokopchuk, and Ramesh Sannareddy, *Hands-on Introduction to Linux Commands and Shell Scripting*, IBM, Coursera.
- William Shotts, *The Linux Command Line*, a free book.
- James Balamuta, *Working with R on a cluster*, a comparison of `Rscript` and `R CMD BATCH`.